MMULT Variants: Nsight Compute Analysis

1. Findings and evidence limits

Best recorded execution times: tile16 with a `(16,16,1)` block for P = 100 x 100; tile32 with a `(32,32,1)` block for P = 1000 x 1000 and P = 2000 x 3000. For the largest case, tile32 takes **30.251040 ms**, a **1.741x speedup** over naive and a **42.56% reduction in kernel time**.

All **31 relevant `.ncu-repz` reports** were opened using the installed Nsight Compute CLI: 16 original MMULT reports and 15 new `NoFFMA-MMULT` reports. Each contains one GPU kernel result. The original set includes three explicitly named naive runs, twelve tiled runs, and one additional naive run misleadingly named `MMULT-Serial-Profile.ncu-repz`; the new set repeats the three naive and twelve tiled configurations with FMA disabled. The mapping from k to matrix dimensions follows the assignment; the recorded launch grids are consistent with that mapping.

New FMA-disabled finding: for the selected largest-case tile32 configuration, duration changes from **30.251040 to 30.413312 ms (+0.54%)**, useful throughput falls **0.53%**, and measured AI changes only **+0.08%**. Occupancy remains approximately 100%. This is a small recorded slowdown, not a 2x penalty or a statistically established difference.

Assignment requirement	What the supplied evidence establishes
CPU serial reference implementation and timing	Not available. The file named Serial profiles `naive_mmult_kernel`, not CPU execution.
Naive and generalized tiled GPU variants	Both kernel families are present; tiled specializations 8, 12, 16, and 32 are recorded. Original CUDA source is not available in this folder, and the inspected tiled report has no source line information. Source-level correctness cannot be certified.
All three matrix-size cases and four tile sizes	Present for the GPU variants.
Release configuration	Recorded executable paths are under `x64\Release`. Compiler flags and the Visual Studio configuration itself are not established by that path alone.
Verification and maximum elementwise error	Not available: there are no output matrices or host verification logs in the supplied evidence.
One combined Roofline plot	Reconstructed below from the recorded metrics, including all 31 GPU results. This is not a screenshot or a saved native Nsight baseline set.
FMA-disabled rebuild and comparison	All 15 NoFFMA reports have zero FFMA and nonzero FADD/FMUL activity. The selected run's SASS confirms separate FMUL/FADD instructions. The exact compiler command is not embedded in these profiles.
Transposed-input experiment	Not supplied; the layout/coalescing tradeoff and break-even condition are discussed in Section 7, without claiming measured transpose timings.

This is a written analysis of existing measurements, not a claim that the original application was modified, rebuilt, verified, or reprofiled. Original profile files were left unchanged.

2. Device, measurement definitions, and combined Roofline

The reports identify an **NVIDIA Tesla T4**, compute capability **7.5**, with **40 SMs**, **16 GB device memory**, and **4 MiB L2 cache**. Recorded limits are 1024 threads/block, 1024 resident threads/SM, 32 resident warps/SM, 16 resident blocks/SM, 65,536 32-bit registers/SM, and up to 64 KiB shared memory/SM. The tile32 run uses a 32 KiB shared-memory configuration, of which each block statically allocates 8 KiB.

The following quantities must not be conflated:

- **Hardware arithmetic intensity (HW AI):** FP32 hardware FLOP/s divided by measured DRAM bytes/s. The numerator follows Nsight's Roofline expression: `(FADD + FMUL + 2*FFMA per cycle) * SMSP clock`. One FFMA counts as **two FLOPs**.
- **Hardware throughput (HW GFLOP/s):** the same hardware-operation rate, including arithmetic on padded tile positions.
- **Useful throughput:** `2*j*k*l / kernel_duration`, using the conventional matrix-multiplication operation count. This is better for comparing time to solve the same problem.
- **SM %:** Nsight's `Compute (SM) Throughput`, an aggregate hardware-resource utilization metric. It is **not** FP32 FLOP/s divided by FP32 peak.
- **Occupancy:** the reported achieved active-warps percentage. Values slightly above 100% are retained as recorded; they are measurement/aggregation effects, not capacity beyond the device limit.

Durations are GPU **kernel durations**, not CPU serial time, transfer time, full application time, or profiler wall-clock time. Each configuration has one recorded launch, so these are recorded rankings, not statistically established averages.

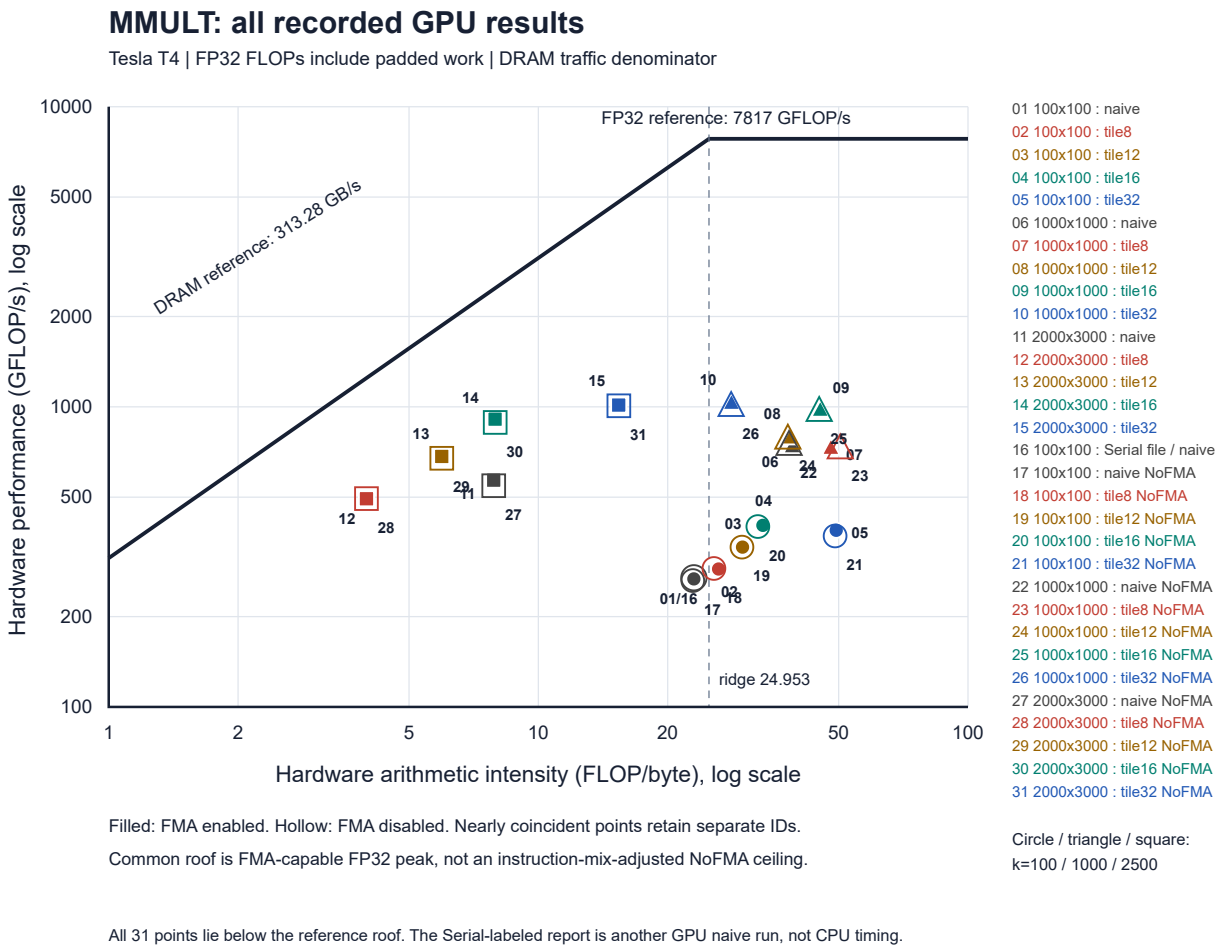


Figure 1. Reconstructed combined FP32 DRAM Roofline. IDs 01-16 are the original FMA-enabled results; IDs 17-31 are NoFFMA results tabulated in Section 6. Color denotes variant, shape denotes matrix-size case, and hollow markers denote NoFFMA. Many paired points nearly coincide. The common FMA-capable reference roof uses the original largest-

case tile32 run: 7817.35 GFLOP/s peak, 313.28 GB/s peak DRAM bandwidth, and ridge AI = 24.953 FLOP/byte. Other runs have different clocks; original-run classification below uses each run's own roof. The NoFFMA instruction-mix ceiling is discussed separately in Section 6.

The reconstruction uses the installed `SpeedOfLight_RooflineChart.section` formulas, rather than assuming AI from tile width. Exported metric values are rounded, so reconstructed rates and AI are approximate. Support files, including all raw/detail metric exports and `summary.csv`, are in `MMULT-analysis`.

Measurements: P = 100 x 100, M = 100 x 100, N = 100 x 100

ID / variant	Block	Duration ms	HW AI FLOP/B	HW GFLOP/s	Useful GFLOP/s	SM %	Occ. %
01 naive	16 x 16	0.007488	23.012	267.09	267.09	30.54	27.25
02 tile8	8 x 8	0.007808	26.273	288.14	256.15	25.69	25.39
03 tile12	12 x 12	0.007392	29.868	340.83	270.56	31.41	31.30
04 tile16	16 x 16	**0.006976**	33.311	**402.79**	**286.70**	30.73	30.54
05 tile32	32 x 32	0.010816	**49.350**	387.79	184.91	27.10	**99.20**
16 Serial label: actually naive	16 x 16	0.007424	23.011	269.39	269.40	30.48	27.15

Measurements: P = 1000 x 1000, M = 1000 x 1000, N = 1000 x 1000

ID / variant	Block	Duration ms	HW AI FLOP/B	HW GFLOP/s	Useful GFLOP/s	SM %	Occ. %
06 naive	16 x 16	2.693184	38.986	742.61	742.62	**74.86**	98.16
07 tile8	8 x 8	2.726208	**47.956**	733.62	733.62	64.17	98.53
08 tile12	12 x 12	2.578368	38.623	794.45	775.68	69.89	92.75
09 tile16	16 x 16	2.085088	45.307	982.40	959.19	73.68	98.70
10 tile32	32 x 32	**2.072640**	28.146	**1036.10**	**964.95**	71.08	**99.85**

Measurements: P = 2000 x 3000, M = 2000 x 2500, N = 2500 x 3000

ID / variant	Block	Duration ms	HW AI FLOP/B	HW GFLOP/s	Useful GFLOP/s	SM %	Occ. %
11 naive	16 x 16	52.661344	7.872	569.67	569.68	64.50	99.03
12 tile8	8 x 8	60.824480	3.980	494.01	493.22	52.91	**100.55**
13 tile12	12 x 12	44.127712	5.958	683.38	679.84	67.83	93.50
14 tile16	16 x 16	33.219776	7.930	909.83	903.08	71.33	99.75
15 tile32	32 x 32	**30.251040**	**15.382**	**1013.52**	**991.70**	**71.37**	100.13

3. Which configurations perform best?

The tables in Sections 2-3 describe the **original FMA-enabled sweep**. The complete NoFFMA sweep and its changed rankings are in Section 6.

Criterion	100 x 100, k=100	1000 x 1000, k=1000	2000 x 3000, k=2500
Highest measured HW AI	tile32: 49.350	tile8: 47.956	tile32: 15.382
Highest FP32 hardware throughput	tile16: 402.79 GFLOP/s	tile32: 1036.10 GFLOP/s	tile32: 1013.52 GFLOP/s
Highest useful throughput / shortest duration	tile16: 6.976 us	tile32: 2.072640 ms	tile32: 30.251040 ms
Highest achieved occupancy	tile32: 99.20%	tile32: 99.85%	tile8: 100.55% reported; tile32 is also approximately full
Highest Compute (SM) Throughput	tile12: 31.41%	naive: 74.86%	tile32: 71.37%
Fastest speedup over naive	1.073x	1.299x	1.741x

The winners depend on what "compute throughput" means: the FP32 FLOP/s winner is not necessarily the aggregate SM-% winner. Highest occupancy and highest AI are also not sufficient to select the fastest kernel.

Are any configurations compute-bound?

In the geometric DRAM-Roofline sense, yes: all four tiled k=100 results are to the right of their FP32 ridge points, and all five k=1000 results, including naive, are to the right. Their relevant ridges are approximately 24.2-25.4 FLOP/byte. The k=100 naive runs are to the left, and **every k=2500 configuration is to the left** of its own ridge, whose value ranges from about 20.4 to 25.0.

However, **being on the compute-ceiling side does not demonstrate that FP32 execution is the actual saturated bottleneck**. These runs achieve only about **3.3-13.0% of their recorded FP32 peaks**, well below the roof. Small grids, instruction issue, cache/shared-memory pipelines, and synchronization can limit performance before either idealized roof is reached. Nsight explicitly diagnoses the small case as grid-underfilled and several larger runs as compute/memory balanced. Likewise, being left of the ridge does not prove that DRAM bandwidth is saturated.

Why do the results change with problem size?

Small case: the grids occupy less than one full device wave. Tile32 has only **16 blocks for 40 SMs**, while tile16 has 49 blocks. Tile32's nearly 100% occupancy describes resident warps while SMs are active; it does not mean all 40 SMs are kept busy. Its padded work is also substantial: 128 x 128 output positions across 128 reduction positions give approximately **2.10x the useful FLOP count**, compared with about 1.40x for tile16. Thus tile32 has the highest HW AI but is the slowest small-case variant. Tile16's lead over naive is only 0.512 us; repeated timings would be needed to establish robustness.

Medium case: there are enough blocks to sustain high occupancy. Tile32 is fastest, but only **0.60% faster than tile16** in the recorded run, so a clear general advantage is not established by one sample. Tile8 has the highest HW AI largely because it incurs only **41.71 MB** of measured DRAM traffic versus **76.30 MB** for tile32, yet tile8 takes longer. Its shorter phases require more synchronization/loading phases, and Nsight reports MIO-pipeline stalls. AI measures DRAM reuse, not all on-chip traffic or instruction costs.

Largest case: the nominal input footprint is **50 MB** and the output is **24 MB**, far beyond the 4 MiB L2. Reuse and traffic differ strongly across launches. Tile8 transfers **7.549 GB**, naive **3.811 GB**, and tile32 **1.993 GB** across DRAM. Tile32 reduces measured DRAM traffic by **47.70% versus naive**, supports high occupancy, and amortizes tile-loading/synchronization overhead across more multiply-adds. It is **8.94% faster than tile16** and the clear recorded winner. Nevertheless, its DRAM throughput is only **21.03% of peak** and its FP32 throughput about **12.97% of peak**; it is not simply saturating external memory or FP32 units.

The largest tile32 run reports **23.6 cycles/warp of MIO-throttle stalls**, about **60.3%** of average cycles between instruction issues, and only **0.97 eligible warps per scheduler per cycle** despite roughly eight active warps. Its recorded shared-memory bank-conflict counters are zero, so bank conflicts should not be asserted as the cause of those stalls.

Recorded clocks also vary: for example, the largest tile8 and tile32 runs have approximately **1.277 GHz** and **1.527 GHz** SM clocks. Rankings describe these runs, not an experiment with identical controlled clocks or repeated confidence intervals.

Thread-block and resource implications

Variant	Threads/block	Registers/thread	Static shared/block	Theoretical occupancy	Maximum resident blocks/SM from recorded limits
naive, 16 x 16	256	50	0 B	100%	4
tile8	64	42	512 B	100%	16
tile12	144	38	1152 B	93.75%	6
tile16	256	40	2048 B	100%	4
tile32	1024	44	8192 B	100%	1

A 144-thread block occupies five 32-thread warps, including a partially filled warp. Six such blocks give 30 of the T4's 32 resident warps, explaining tile12's 93.75% theoretical occupancy. Tile32 reaches the maximum 1024 threads/block and permits only one block/SM. It fits the register/shared-memory limits, but low grid-block count can leave SMs idle for small problems.

The tested experiments tie tile width to square block width. There is **no evidence of an independent block-shape sweep** beyond `(8,8,1)`, `(12,12,1)`, `(16,16,1)`, and `(32,32,1)`; naive always uses `(16,16,1)`.

4. General MMULT implementations and verification

For row-major matrices, the required operation is:

```
P[row*l + col] = sum(M[row*k + q] * N[q*l + col], q=0..k-1)
```

The CPU reference uses three serial loops. The naive GPU variant assigns one output element to each thread and obtains operands from global memory. The tiled GPU variant cooperatively stages two T x T tiles in shared memory, processes `ceil(k/T)` phases, and zero-fills out-of-range loads. Both synchronization barriers must be reached by **all threads**, including threads outside the output boundary; only the final store is conditional.

The following is **reference implementation guidance**, not source recovered from, or tested against, the supplied reports. It requires the usual CUDA runtime and C++ headers. Arrays must be correctly sized and nonaliasing; host and device arrays must be allocated/copied by the application.

```
void cpu_mmult(const float* M, const float* N, float* P,
               int j, int k, int l) {
    for (int row = 0; row < j; ++row)
        for (int col = 0; col < l; ++col) {
            float sum = 0.0f;
            for (int q = 0; q < k; ++q)
```

```
                sum += M[size_t(row) * k + q] *
                       N[size_t(q) * l + col];
            P[size_t(row) * l + col] = sum;
        }
}

__global__ void naive_mmult(const float* M, const float* N, float* P,
                            int j, int k, int l) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= j || col >= l) return;
    float sum = 0.0f;
    for (int q = 0; q < k; ++q)
        sum += M[size_t(row) * k + q] * N[size_t(q) * l + col];
    P[size_t(row) * l + col] = sum;
}

template<int T>
__global__ void tiled_mmult(const float* M, const float* N, float* P,
                            int j, int k, int l) {
    __shared__ float A[T][T], B[T][T];
    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y * T + ty, col = blockIdx.x * T + tx;
    float sum = 0.0f;
    for (int phase = 0; phase < (k + T - 1) / T; ++phase) {
        int aq = phase * T + tx, bq = phase * T + ty;
        A[ty][tx] = (row < j && aq < k)
            ? M[size_t(row) * k + aq] : 0.0f;
        B[ty][tx] = (bq < k && col < l)
            ? N[size_t(bq) * l + col] : 0.0f;
        __syncthreads();
        for (int q = 0; q < T; ++q)
            sum += A[ty][q] * B[q][tx];
        __syncthreads();
    }
    if (row < j && col < l)
        P[size_t(row) * l + col] = sum;
}
```

For naive, use `block=(16,16,1)` and `grid=(ceil(l/16),ceil(j/16),1)`. For tiled specialization T, use `block=(T,T,1)` and `grid=(ceil(l/T),ceil(j/T),1)`. All four requested tile widths fit this T4. Test nonsquare dimensions and tails in **all three dimensions**, not just the output dimensions.

Host verification and the maximum error question

The maximum observed single-element absolute error fraction is unknown for every supplied case. Nsight kernel-performance counters do not contain the output values required to calculate it. Neither zero error nor successful verification can be inferred from these reports.

Output size / k	Naive maximum error fraction	Tiled maximum error fraction
100 x 100 / 100	Not recorded	Not recorded for T=8,12,16,32
1000 x 1000 / 1000	Not recorded	Not recorded for T=8,12,16,32
2000 x 3000 / 2500	Not recorded	Not recorded for T=8,12,16,32

For a nonzero reference, compute `abs(cpu-gpu)/abs(cpu)`. For the positive CPU values implicit in the assignment's expression, this equals its stated `abs(cpu-gpu)/cpu`. The absolute denominator also handles negative references sensibly. A zero reference requires an absolute tolerance, because its relative error is undefined.

The following host helper checks finite values, applies a mixed tolerance, and reports the requested maximum relative fraction. **Illustrative starting tolerances** are `rtol=1e-4` and `atol=1e-5` for unit-scale, nonnegative FP32 test inputs; they are not established acceptance thresholds for arbitrary inputs or cancellation-heavy data.

```
struct Check {
    bool pass;
    double max_abs;
    double max_fraction; // Over nonzero reference elements.
    bool has_nonzero_reference;
};

Check verify(const std::vector<float>& cpu,
             const std::vector<float>& gpu,
             double rtol = 1e-4, double atol = 1e-5) {
    if (cpu.size() != gpu.size())
        throw std::invalid_argument("Output sizes differ");
    Check out{true, 0.0, 0.0, false};
    for (size_t i = 0; i < cpu.size(); ++i) {
        double ref = cpu[i], got = gpu[i];
        if (!std::isfinite(ref) || !std::isfinite(got)) {
            out.pass = false;
            out.max_abs = out.max_fraction =
                std::numeric_limits<double>::infinity();
            continue;
        }
        double error = std::abs(ref - got);
        out.max_abs = std::max(out.max_abs, error);
        out.pass &= error <= atol + rtol * std::abs(ref);
        if (ref != 0.0) {
            out.has_nonzero_reference = true;
            out.max_fraction = std::max(out.max_fraction,
                                        error / std::abs(ref));
        }
    }
    return out;
}
```

Use the **same input matrices** for CPU, naive, and every tiled run. For each of the three sizes, compute the CPU reference once, launch naive and each tiled specialization, check CUDA launch/copy/synchronization errors, copy each result back, and call `verify(cpu,gpu)`. Also call `verify(naive,tiled)` if a direct pairwise check is desired. Log maximum absolute error, maximum fraction over nonzero references, and pass/fail for each comparison; report the fraction as a fraction, or multiply by 100 if labeling it as a percentage. An all-zero reference has no defined maximum relative fraction and should be labeled accordingly. FMA changes rounding, so bitwise equality is not required.

This explains how to fulfill the implementation/verification requirements; the supplied profiles alone do not demonstrate that the original host application fulfilled them.

5. Release profiling and native baselines

The recorded session command for the largest tile32 run includes `--set full` and launches `x64\Release\Homework-3-MMULT.exe -tiled -k2500 -tile32`. The Serial-labeled report instead records `-naive -k100`. Nsight Compute profiles GPU launches; CPU timing must be collected separately with a host timer.

To reproduce the assignment's **native GUI baseline workflow**, open each existing report, select its kernel result, add it as a baseline using the baseline control, and assign descriptive names such as `P2000x3000_k2500_tile32_block32x32_FMA`. Repeat for the other variants and cases, enable those baselines, and inspect **GPU Speed Of Light -> Roofline**. Label the Serial-file entry as an additional naive k=100 run, not CPU serial. Figure 1 supplies the combined comparison without claiming that the UI baseline list or original binary reports were edited.

6. Disabling FMA for the selected largest-case configuration

Select **TILE_WIDTH=32, block=(32,32,1)** for P = 2000 x 3000, k = 2500, based on the fastest original kernel duration. The new matching report is `NoFFMA-MMULT-tiled-k2500-tile32--Profile.ncu-repz`.

Metric	FMA enabled (ID 15)	FMA disabled (ID 31)	Change
HW AI	15.381785 FLOP/B	15.394078 FLOP/B	+0.08%
Duration	30.251040 ms	30.413312 ms	+0.162272 ms; +0.54%
FP32 hardware throughput	1013.52 GFLOP/s	1008.12 GFLOP/s	-0.53%
Useful throughput	991.70 GFLOP/s	986.41 GFLOP/s	-0.53%
Compute (SM) Throughput	71.37%	70.35%	-1.02 percentage points
Achieved occupancy	100.13% reported	99.99%	-0.14 percentage points; both approximately full
Theoretical occupancy	100%	100%	Unchanged
Registers/thread	44	40	-4; no occupancy-tier change
Static shared memory/block	8192 bytes	8192 bytes	Unchanged
DRAM bytes read + written	1,993,273,184	1,991,701,696	-0.08%
DRAM throughput	65.891 GB/s	65.488 GB/s	-0.61%
FFMA / FADD / FMUL per elapsed cycle	331.91 / 0 / 0	0 / 327.15 / 327.15	Fused arithmetic replaced by separate operations
Executed instructions (warp-level)	1,504,273,728	1,983,339,840	+31.85% across the whole kernel

Yes, recorded performance changed, but only slightly: disabling FMA increased duration by 0.54%, rather than doubling it. The floating-point arithmetic instruction count doubles for the multiply-add work, but the total kernel instruction count rises only 31.85%, because loads, stores, indexing, branches, and synchronization are also present. The nominal launch is unchanged: 5922 blocks, 1024 threads/block, and 148.05 waves/SM.

The original run was not near its FP32 ceiling and already had substantial MIO-pipeline stalls. The NoFFMA run still reports MIO throttling: **17.5 cycles/warp, 58.2% of cycles between issues**, versus 23.6 cycles and 60.3% originally. Eligible warps/scheduler rise from **0.97 to 1.12**. These measurements are consistent with instruction scheduling and on-chip memory pressure limiting the kernel, rather than a pure multiply-add-throughput bottleneck; stall cycles per instruction are not total stall time and cannot alone prove a causal improvement. Splitting arithmetic can therefore add instructions without adding a proportional kernel-time penalty.

The runs are not controlled repeated trials: their recorded SM clocks are approximately **1.527 and 1.541 GHz**, and there is only one recorded launch per configuration. A 0.54% duration difference is too small to establish a reproducible compiler-option penalty from this evidence alone. The conclusion is **nearly unchanged observed performance**, not that disabling FMA is generally free or beneficial. Register allocation actually decreases here, so the hypothetical increased-register/occupancy penalty does not occur.

All 15 NoFFMA reports have **zero FFMA** and equal, nonzero FADD/FMUL rates in the FP32 counters. The selected report's SASS also contains separate `FMUL` and `FADD`. Its recorded command launches `x64\Release\Homework-3-MMULT.exe -tiled -k2500 -tile32`. This establishes unfused FP32 execution in the measured kernels, although the original project settings/compiler invocation are not available to independently certify the precise flag used.

For reproducibility, the requested setting is Visual Studio **Release -> Configuration Properties -> CUDA C/C++ -> Command Line -> Additional Options:** `--fmad=false` (the compiler default is `true`), followed by a rebuild and profiling of the same arguments. Add the imported NoFFMA runs as separate baselines, with names such as `P2000x3000_k2500_tile32_block32x32_NoFMA`. No application rebuild or new GPU profiling was performed for this report update.

Complete new NoFFMA sweep

Each block width equals TILE_WIDTH for tiled kernels; naive retains a 16 x 16 block. Duration change is relative to the matching original FMA-enabled report; a positive change means slower.

Plot ID / size / variant	Duration ms	Change %	HW AI FLOP/B	HW GFLOP/s	Useful GFLOP/s	SM %	Occ. %
17 / 100 x 100 / naive	0.007584	+1.28	22.878	263.72	263.71	30.43	27.50
18 / 100 x 100 / tile8	0.007776	-0.41	25.622	289.33	257.20	25.91	25.50
19 / 100 x 100 / tile12	0.007392	0.00	29.776	340.82	270.56	29.96	30.01
20 / 100 x 100 / tile16	0.007040	+0.92	32.424	399.11	284.09	30.09	30.65
21 / 100 x 100 / tile32	0.011296	+4.44	49.053	371.30	177.05	26.34	99.43
22 / 1000 x 1000 / naive	2.650560	-1.58	38.330	754.56	754.56	76.22	98.17
23 / 1000 x 1000 / tile8	2.722272	-0.14	50.433	734.68	734.68	64.40	98.56
24 / 1000 x 1000 / tile12	2.586976	+0.33	38.079	791.80	773.10	69.61	92.73
25 / 1000 x 1000 / tile16	2.091840	+0.32	45.044	979.24	956.10	73.39	98.66
26 / 1000 x 1000 / tile32	2.111296	+1.87	28.101	1017.15	947.29	69.88	99.95
27 / 2000 x 3000 / naive	54.943936	+4.33	7.872	546.00	546.01	64.85	99.05
28 / 2000 x 3000 / tile8	60.760960	-0.10	3.980	494.54	493.74	53.12	98.55
29 / 2000 x 3000 / tile12	44.752416	+1.42	5.959	673.84	670.35	67.76	93.33
30 / 2000 x 3000 / tile16	34.068768	+2.56	7.935	887.15	880.57	71.16	100.36
31 / 2000 x 3000 / tile32	30.413312	+0.54	15.394	1008.12	986.41	70.35	99.99

With FMA disabled, **tile16 is fastest for both the small and medium cases**, and tile32 remains fastest for the largest case. The medium-case tile16/tile32 ordering reverses, but their new durations differ by only 0.92%, again requiring repeated trials before treating the reversal as robust. Highest AI remains tile32, tile8, tile32 respectively; highest HW GFLOP/s remains tile16, tile32, tile32. For the medium case, tile32's larger padded arithmetic count explains why it can have higher HW GFLOP/s yet a longer duration than tile16. Highest SM-% is naive, naive, tile16 respectively; highest recorded achieved occupancy is tile32, tile32, tile16 respectively. Differences around 100% do not establish meaningful occupancy advantages.

What should happen to arithmetic intensity?

An FMA computes a multiply and an add in **one instruction but two floating-point operations**. Separate FMUL and FADD compute the same **two FLOPs in two instructions**. Therefore:

```
With FMA:       2 FLOPs / B bytes
Without FMA:    (1 MUL FLOP + 1 ADD FLOP) / B bytes
```

FLOP-based algorithmic AI does not double or halve merely because FMA is disabled. It remains the same if executed arithmetic and memory traffic remain unchanged. Floating-point **instruction** count increases, but instructions/byte is not the FLOP/byte metric used by this Roofline. Measured HW AI can still shift if the compiler changes executed operations, register allocation, spilling, or cache/DRAM traffic.

The selected pair illustrates this directly: multiplying HW GFLOP/s by duration gives approximately **30.66 billion executed FP32 FLOPs in each run**, including padding. DRAM traffic changes by only -0.08%, accounting for the approximately +0.08% measured AI shift. The algorithm is not doing twice the useful arithmetic.

The chart retains the usual **FMA-capable FP32 roof** to compare all baselines consistently. For a balanced separate-MUL/ADD stream on the same FP32 execution resources, an idealized instruction-mix ceiling is approximately **half** that FMA FLOP/s peak. For the selected NoFFMA run, that is approximately **3944 GFLOP/s**, with a DRAM ridge near **12.045 FLOP/B** instead of 24.090 using that run's full FMA-capable roof. Its AI of 15.394 is therefore on the compute side of this **mix-adjusted** ridge, although its approximately 1008 GFLOP/s is still far below even that reduced ceiling. This changes the geometric classification, not the evidence for an actually saturated FP32 bottleneck. Against each run's conventional FMA-capable roof, the new runs have the same left/right classification pattern as the original sweep.

The general tiled algorithm's idealized tile-level global-memory AI is approximately **T/4 FLOP/byte** for FP32, neglecting output stores and edge overhead: `2*T^3 FLOPs / (2*T^2*4 bytes)`. Naive is approximately 0.25 FLOP/byte under a no-reuse load model. These are **not** the measured DRAM AIs in the tables: caches, cross-block reuse, padding, and actual traffic materially change the hardware metric. Neither idealized expression changes solely from disabling FMA.

7. Additional assignment questions

Is transposing N worth its overhead?

Not established by these profiles: there is no transpose kernel, transposed-layout MMULT run, or end-to-end timing comparison. The reference indexing in Section 4 already assumes both M and N are stored row-major. Reading N down a column in one thread does not mean N itself is column-major.

For the given one-output-per-thread mapping, adjacent x-lanes read adjacent `N[q*l + col]` values at fixed q, so the row-major N accesses are coalesced across the warp. Physically transposing N and preserving the same mathematical product requires indexing `N_T[col*k + q]`; at fixed q, adjacent lanes would then be k floats apart. Thus transposition alone can **worsen** coalescing for this mapping. The tiled reference similarly loads contiguous rows cooperatively and reuses N in shared memory; a global transpose is not automatically beneficial. A redesigned kernel or a genuinely different original storage layout could change that conclusion, but neither is demonstrated by the available source-free reports. Merely transposing without correcting dimensions/indexing changes the operation, not just its performance.

The correct break-even test for one multiplication is:

```
transpose_time + transposed_layout_mmult_time < original_mmult_time
```

For R multiplications reusing the same transposed N, compare `transpose_time/R + transposed_layout_mmult_time` with the original time. For the largest case, N has 7.5 million FP32 elements (30 MB), so a materialized transpose must at least read and write **60 MB** and allocate another 30 MB. At the original tile32 roof's 313.28 GB/s, that traffic alone has an optimistic lower bound of approximately **0.192 ms**, excluding launch, layout inefficiency, and other overheads. This is a bandwidth model, not a measured transpose time. Without a measured MMULT improvement greater than the actual transpose cost, the overhead is not justified; reuse across many products can make it worthwhile if the redesigned access pattern actually helps.

Does a 32 x 32 block guarantee highest occupancy?

No. 1024 threads/block is a launch limit, while occupancy is resident active warps relative to the SM's warp capacity. Registers, shared memory, warp/block allocation granularity, device-specific resident-thread limits, and available blocks all matter. On this T4, one tile32 block uses all 32 resident warp slots and fits its resources, so theoretical occupancy is 100%; tile8 and tile16 can also reach 100% using multiple resident blocks. Tile12 reaches 93.75% because six five-warp blocks fill only 30 warp slots.

For small inputs, tile32 launches only 16 blocks across 40 SMs: its approximately 99% achieved occupancy does not mean device-wide coverage is full, and it is slower than tile16. A register/shared-memory demand too large to fit one 1024-thread block would make that launch invalid, not guarantee occupancy. On GPUs supporting more than 1024 resident threads/SM, one such block may also leave warp capacity unused if a second block cannot fit. Even high occupancy does not guarantee many **eligible** warps or high performance, as the selected largest-case scheduling metrics show.

8. Sources and traceability

Primary evidence is the original local reports:

Plot IDs	Report files
01, 06, 11	`MMULT-naive-k100--Profile.ncu-repz`, `MMULT-naive-k1000--Profile.ncu-repz`, `MMULT-naive-k2500--Profile.ncu-repz`
02-05	`MMULT-tiled-k100-tile8--Profile.ncu-repz`, `MMULT-tiled-k100-tile12--Profile.ncu-repz`, `MMULT-tiled-k100-tile16--Profile.ncu-repz`, `MMULT-tiled-k100-tile32--Profile.ncu-repz`
07-10	The corresponding four `MMULT-tiled-k1000-tile...--Profile.ncu-repz` reports
12-15	The corresponding four `MMULT-tiled-k2500-tile...--Profile.ncu-repz` reports
16	`MMULT-Serial-Profile.ncu-repz`
17, 22, 27	The three corresponding `NoFFMA-MMULT-naive-k...--Profile.ncu-repz` reports
18-21	The four `NoFFMA-MMULT-tiled-k100-tile...--Profile.ncu-repz` reports, tile8/12/16/32
23-26	The four corresponding `NoFFMA-MMULT-tiled-k1000-tile...--Profile.ncu-repz` reports
28-31	The four corresponding `NoFFMA-MMULT-tiled-k2500-tile...--Profile.ncu-repz` reports

All report data were imported with `ncu --import <report> --page raw --csv --print-units base` and `--page details --csv --print-units base`. Session/device metadata and original/NoFFMA tile32 SASS were also inspected; the new SASS export is retained as `MMULT-analysis\NoFFMA-MMULT-tiled-k2500-tile32.sass.txt`. Calculations are reproducible

with `MMULT-analysis\extract-metrics.ps1`; all 31 raw/detail export pairs and derived values, including an explicit enabled/disabled FMA label, are retained in that directory. `MMULT-analysis\render-report.ps1` regenerates the combined SVG and the HTML used for the PDF.

Roofline expressions were checked against `C:\Program Files\NVIDIA Corporation\Nsight Compute 2026.4.0\sections\SpeedOfLight_RooflineChart.section`. Definitions and interpretation: NVIDIA Nsight Compute Profiling Guide: Roofline. Compiler option: NVIDIA CUDA Compiler Driver: --fmad.